

FPT University
HO CHI MINH CITY CAMPUS



FPT UNIVERSITY

Linux and Open Source Platform

Lab 1:

Linux Kernel Configuration & System Boot (ARMv7)

Name	Roll number	Subject
Hoàng Ngọc Gia Bảo	SE203238	LOS201
Trần Hồ Khánh Tân	SE203740	LOS201
Nguyễn Trọng Hùng	SE204352	LOS201
Nguyễn Quế Vũ	SE204290	LOS201
Nguyễn Việt Anh	SE204364	LOS201
Hồ Phan Anh Tuấn	SE203437	LOS201

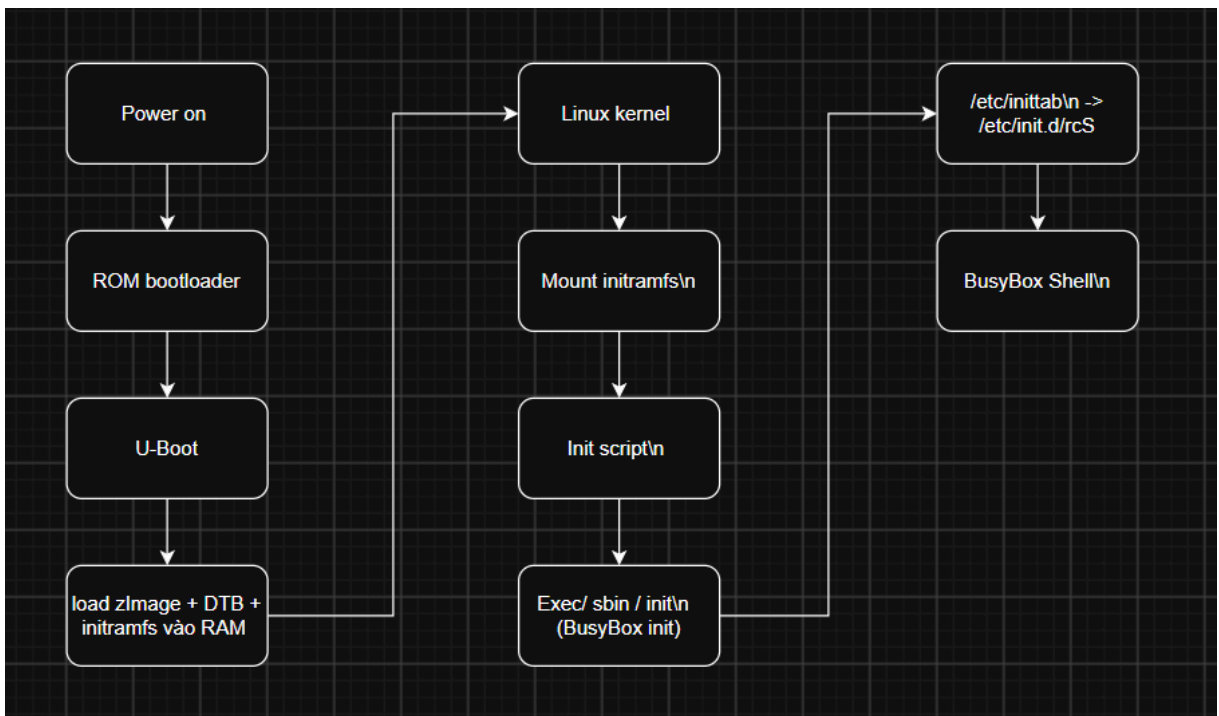
Lecturer: Phạm Thế Vinh

Ho Chi Minh City, Sep 15, 2026

Experimental Setup and Technical Specifications

- **Host Operating System:** Ubuntu 22.04 LTS (x86_64).
- **Cross-Compiler Toolchain:** `arm-linux-gnueabi-hf-gcc` (supporting ARMv7 architecture with hardware floating-point and NEON).
- **Target Emulated Hardware:** ARM Versatile Express Cortex-A9 (`vexpress-a9`).
- **System Memory (RAM):** 512 MB.
- **Emulator:** QEMU ARM (`qemu-system-arm` v7.x / v8.x).
- **Software Components:**
 - **Bootloader:** Das U-Boot v2022.04.
 - **Linux Kernel:** Linux v5.15 LTS.
 - **Root Filesystem:** BusyBox v1.35.0.

1. Boot Sequence Overview



1.1 Boot Sequence Steps

Step	Component	Main task	What is passed to the next step?
1	Power ON	Power supplied to the board (QEMU emulation)	CPU begins executing from reset vector
2	ROM Bootloader	Fixed code in ROM, initializes basic hardware	Jumps to U-Boot address in Flash/RAM
3	U-Boot	Initializes DRAM, console, sets up bootargs & bootcmd	Loads zImage, DTB, initramfs into RAM; passes bootargs to

			kernel
4	Load Images	U-Boot loads zImage, DTB, initramfs	Calls bootz to transfer control to kernel
5	Linux Kernel	Decompresses zImage, runs start_kernel(), initializes scheduler, drivers, memory management	Mounts root filesystem, execs init process
6	Mount initramfs	Decompresses initramfs from RAM into rootfs (tmpfs)	Provides root filesystem for userspace
7	/init script	Mounts /proc, /sys then exec /sbin/init	Transfers control to BusyBox init
8	BusyBox init	Reads /etc/inittab, runs /etc/init.d/rcS	Starts system services
9	BusyBox Shell	Displays shell prompt for the user	System is ready for use

2. Kernel Configuration and Compilation

```

viet-anh@viet-anh-Thin-A15-B7UCX:~/embedded_lab1/kernel$ cd ~/embedded_lab1/kernel/linux-5.15
make vexpress_defconfig
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/kconfig/conf.o
HOSTCC  scripts/kconfig/confdata.o
HOSTCC  scripts/kconfig/expr.o
LEX      scripts/kconfig/lexer.lex.c
YACC     scripts/kconfig/parser.tab.[ch]
HOSTCC  scripts/kconfig/lexer.lex.o
HOSTCC  scripts/kconfig/menu.o
HOSTCC  scripts/kconfig/parser.tab.o
HOSTCC  scripts/kconfig/preprocess.o
HOSTCC  scripts/kconfig/symbol.o
HOSTCC  scripts/kconfig/util.o
HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
viet-anh@viet-anh-Thin-A15-B7UCX:~/embedded_lab1/kernel/linux-5.15$

```

- Loads the default hardware configuration (**defconfig**) specifically tailored for the ARM Versatile Express platform supporting Cortex-A9 processors, automatically generating the baseline **.config** file.

2.1 Build Kernel

```

oc) KCFLAGS="-march=armv7-a" zImage dtbs modules
CALL    scripts/atomic/check-atomics.sh
CALL    scripts/checksyscalls.sh
CHK     include/generated/compile.h
Kernel: arch/arm/boot/Image is ready
Kernel: arch/arm/boot/zImage is ready
viet-anh@viet-anh-Thin-A15-B7UCX:~/embedded_lab1/kernel/linux-5.15$

```

- Compiles the Linux kernel for the ARMv7 architecture using multi-core parallel processing (`-j$(nproc)`), applying explicit target architecture flags via `KCFLAGS` to build the compressed kernel image (`zImage`), Device Tree Blobs (`.dtb`), and kernel modules.

2.2 Output Verification of `zImage`

```

Kernel: arch/arm/boot/zImage is ready
viet-anh@viet-anh-Thin-A15-B7UCX:~/embedded_lab1/kernel/linux-5.15$ ls -lh arch/arm/boot/zImage
ls -lh arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 viet-anh viet-anh 4.9M Sep 20 16:52 arch/arm/boot/zImage
-rw-rw-r-- 1 viet-anh viet-anh 14K Sep 20 16:50 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
viet-anh@viet-anh-Thin-A15-B7UCX:~/embedded_lab1/kernel/linux-5.15$

```

- Lists the compiled kernel image file in long format with human-readable file sizes to verify its successful generation and check its storage footprint.

3. System Boot Execution & Testing

3.1 Kernel Panic without Root Filesystem

```
tan@tantran: ~/embedded_lab1/kernel/linux-5.15 -- qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -nographic -kernel /home/tan/embedded_lab1/output/zimag...
Sep 15 20:45
ledtrig-cpu: registered to indicate activity on CPUs
usbcore: registered new interface driver usbhid
usbhid: USB HID core driver
NET: Registered PF_PACKET protocol family
ipnet: installing IP2200 support
Registering SWP/SWPB emulation handler
aaci-pl041 10004000.aaci: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
aaci-pl041 10004000.aaci: FIFO 312 entries
nvci-pl18x 10005000.nvci: Got CD GP10
nvci-pl18x 10005000.nvci: Got MP GP10
nvci-pl18x 10005000.nvci: nvci0: PL181 manf 41 rev0 at 0x10005000 (irq 33,34 (pio))
10005000.uart: ttyAMA0 at MMIO 0x10005000 (irq = 37, base_baud = 0) is a PL811 rev1
prink: console (ttyAMA0) enabled
1000a000.uart: ttyAMA1 at MMIO 0x1000a000 (irq = 38, base_baud = 0) is a PL811 rev1
1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL811 rev1
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL811 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2025-09-15T13:44:29 UTC (170479669)
amba 10020000.cldc: Fixing up cyclic dependency with 0-0039
drn-clcd-pl111 1001f000.cldc: DVI muxed to daughterboard 1 (core tile) CLCD
drn-clcd-pl111 1001f000.cldc: Initializing Versatile Express PL111
sti902x 0-0039: supply iovcc not found, using dummy regulator
sti902x 0-0039: supply cvcc12 not found, using dummy regulator
l2c-l2c-0: Added multiplexed l2c bus 2
drn-clcd-pl111 1001f000.cldc: assigned reserved memory node vram@4c000000
drn-clcd-pl111 1001f000.cldc: using device-specific reserved memory
drn-clcd-pl111 1001f000.cldc: core tile graphics present
drn-clcd-pl111 1001f000.cldc: this device will be deactivated
drn-clcd-pl111 1001f000.cldc: Versatile Express unit failed - -19
drn-clcd-pl111 10020000.cldc: DVI moved to daughterboard 1 (core tile) CLCD
drn-clcd-pl111 10020000.cldc: Initializing Versatile Express PL111
drn-clcd-pl111 10020000.cldc: found bridge on endpoint 0
drn-clcd-pl111 10020000.cldc: Using non-panel bridge
[drm] Initialized pl111 i.0.0 20170317 for 10020000.cldc on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000/notherboard-bus@40000000/lofpga7,00000000/10007000.lvi/serio0/lnput/lnput0
console: switching to colour frame buffer device 128x40
drn-clcd-pl111 10020000.cldc: [drm] fb0: pl111 frame buffer device
ALSA device list:
#0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
input: Inexp/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000/notherboard-bus@40000000/lofpga7,00000000/10007000.lvi/serio1/lnput/lnput2
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00      131072 mtbblock0
(driver?)
1f01      32768 mtbblock1
(driver?)
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
<00183a9> (unwind backtrace) from [<0018c2c4>] (show_stack+0x10/0x14)
<0018c2c4> (show_stack) from [<00067f0b>] (dump_stack+0x10/0x1c)
<00067f0b> (dump_stack) from [<00063f64>] (panic+0x1c/0x2e4)
<00063f64> (panic) from [<00063f64>] (panic+0x1c/0x2e4)
<00063f64> (panic) from [<00063f64>] (panic+0x1c/0x2e4)
<00063f64> (panic) from [<00063f64>] (panic+0x1c/0x2e4)
<00063f64> (panic) from [<00063f64>] (panic+0x1c/0x2e4)
<00063f64> (panic) from [<00063f64>] (panic+0x1c/0x2e4)
Exception stack(0x010a7f00 to 0x010a7ffb)
7fa0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fc0: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
7fed: 00000000 00000000 00000000 00000000 00000000 00000000 00000000 00000000
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

-The screenshot illustrates a Kernel Panic event triggered during the boot process of the Linux kernel on the QEMU vexpress-a9 platform. This occurs because the kernel successfully initializes the hardware drivers, serial console (ttyAMA0), and drivers, but fails at the final stage of the initialization sequence when attempting to mount the root file system (VFS: Unable to mount root fs on unknown-block(0,0)). Specifically, since the initial RAM disk (initramfs) or rootfs image was omitted from the QEMU command line during this test run, the kernel cannot find a valid storage device or partition to hand over control to the first userspace process (/sbin/init), resulting in a system crash ("Kernel panic - not syncing"). This expected behavior demonstrates the crucial dependency of the Linux boot sequence on a properly packaged root filesystem

3.2 Configuring Boot Arguments in U-Boot

-The screenshot presents the comprehensive output of the `printenv` command within the U-Boot environment on the QEMU vexpress-a9 platform. This listing reveals all default and customized environment variables configured for the bootloader. Key details include standard input, output, and error redirection to the serial interface (`stdin=serial`, `stdout=serial`, `stderr=serial`), and the definition of networking parameters such as the board's MAC address (`ethaddr=52:54:00:12:34:56`). Additionally, it outlines complex automated boot execution strings like `bootcmd` and `bootdelay=2`, which manages the initialization countdown window before launching the operating system.

3.4 Successful System Boot with Initramfs

1. *ls initramfs/ output log*

```

anh Tuan@Acer-Nitro5-AN-515-58:~/embedded_lab1/rootfs/busybox-1.35.0/_install$ cd ~/embedded_lab1/rootfs/busybox-1.35.0/_install
ls -la
total 44
drwxrwxr-x 11 anh Tuan anh Tuan 4096 Sep 13 21:21 .
drwxr-xr-x 37 anh Tuan anh Tuan 4096 Sep 14 14:20 ..
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:20 bin
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:21 dev
drwxrwxr-x 3 anh Tuan anh Tuan 4096 Sep 13 21:22 etc
lrwxrwxrwx 1 anh Tuan anh Tuan 11 Sep 13 21:20 linuxrc -> bin/busybox
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:21 proc
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:21 root
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:20 sbin
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:21 sys
drwxrwxr-x 2 anh Tuan anh Tuan 4096 Sep 13 21:21 tmp
drwxrwxr-x 4 anh Tuan anh Tuan 4096 Sep 13 21:20 usr
anh Tuan@Acer-Nitro5-AN-515-58:~/embedded_lab1/rootfs/busybox-1.35.0/_install$ ls -lh ~/embedded_lab1/output/
total 10M
-rw-rw-r-- 1 anh Tuan anh Tuan 1.1M Sep 14 14:22 initramfs.cpio.gz
-rwxrwxr-x 1 anh Tuan anh Tuan 4.2M Sep 14 14:04 u-boot
-rw-rw-r-- 1 anh Tuan anh Tuan 14K Sep 14 14:03 vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 anh Tuan anh Tuan 4.8M Sep 14 14:03 zImage
anh Tuan@Acer-Nitro5-AN-515-58:~/embedded_lab1/rootfs/busybox-1.35.0/_install$

```

-The initial RAM filesystem was created as the root filesystem for the Embedded Linux System. It contains the directories and file required for the system to start the userspace environment. The output log showed the directory structure including directories such as `bin`, `dev`, `ect`, `proc`, `root`, `sbin`, `sys`, `tmp`, and `usr`, which means the root filesystem has been prepared successfully for the embedded Linux system.

2. *Qemu booted successfully*

```

1000b000.uart: ttyAMA2 at MMIO 0x1000b000 (irq = 39, base_baud = 0) is a PL011 rev1
1000c000.uart: ttyAMA3 at MMIO 0x1000c000 (irq = 40, base_baud = 0) is a PL011 rev1
rtc-pl031 10017000.rtc: registered as rtc0
rtc-pl031 10017000.rtc: setting system clock to 2026-09-16T11:10:47 UTC (1789557047)
amba 10020000.clcd: Fixing up cyclic dependency with 0-0039
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: Initializing Versatile Express PL111
sii902x 0-0039: supply iovcc not found, using dummy regulator
sii902x 0-0039: supply cvcc12 not found, using dummy regulator
i2c i2c-0: Added multiplexed i2c bus 2
drm-clcd-pl111 1001f000.clcd: assigned reserved memory node vram@4c000000
drm-clcd-pl111 1001f000.clcd: using device-specific reserved memory
drm-clcd-pl111 1001f000.clcd: core tile graphics present
drm-clcd-pl111 1001f000.clcd: this device will be deactivated
drm-clcd-pl111 1001f000.clcd: Versatile Express init failed - -19
drm-clcd-pl111 10020000.clcd: DVI muxed to daughterboard 1 (core tile) CLCD
drm-clcd-pl111 10020000.clcd: Initializing Versatile Express PL111
drm-clcd-pl111 10020000.clcd: found bridge on endpoint 0
drm-clcd-pl111 10020000.clcd: Using non-panel bridge
[drm] Initialized pl111 1.0.0 20170317 for 10020000.clcd on minor 0
input: AT Raw Set 2 keyboard as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:iopfpga@7,00000000/10006000.kmi/ser
i00/input/input0
Console: switching to colour frame buffer device 128x48
drm-clcd-pl111 10020000.clcd: [drm] fb0: pl111 frame buffer device
ALSA device list:
 #0: ARM AC'97 Interface PL041 rev0 at 0x10004000, irq 32
Freeing unused kernel image (initmem) memory: 1024K
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1

Please press Enter to activate this console. input: InExPS/2 Generic Explorer Mouse as /devices/platform/bus@40000000/bus@40000000:motherboard-bus@40000000/bus@40000000:iofpga@7,00000000/10007000.kmi/serio1/input/input2
random: fast init done

/ #

```

-The Linux system was booted using QEMU with the ARM Versatile Express A9 model, and Cortex-A9 CPU. The kernel, dtb, and compressed initramfs were provided to QEMU. The boot log showed that Linux kernel version 5.15.0 was successfully loaded and the system was running on ARMv7 processor. The appearance of the `/ #` prompt confirmed that the Linux kernel, initramfs, BusyBox, and userspace initialization process were all working successfully.

3.uname -a, mount, and ps

```

random: fast init done

/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sun Sep 13 20:10:27 +07 2026 armv7l GNU/Linux
/ # mount
rootfs on / type rootfs (rw,size=244912k,nr_inodes=61228)
none on /proc type proc (rw,relative)
none on /sys type sysfs (rw,relative)
none on /tmp type tmpfs (rw,relative)
/ # ps
PID  USER    TIME  COMMAND
  1  root      0:02  init
  2  root      0:00  [kthreadd]
  3  root      0:00  [rcu_gp]
  4  root      0:00  [rcu_par_gp]
  5  root      0:00  [kworker/0:0-eve]
  6  root      0:00  [kworker/0:0H]
  7  root      0:00  [kworker/u8:0-ev]
  8  root      0:00  [mm_percpu_wq]
  9  root      0:00  [ksoftirqd/0]
 10  root      0:00  [rcu_sched]
 11  root      0:00  [migration/0]
 12  root      0:00  [cpuhp/0]
 13  root      0:00  [cpuhp/1]
 14  root      0:00  [migration/1]
 15  root      0:00  [ksoftirqd/1]
 16  root      0:00  [kworker/1:0-rcu]
 17  root      0:00  [kworker/1:0H-ev]
 18  root      0:00  [cpuhp/2]
 19  root      0:00  [migration/2]
 20  root      0:00  [ksoftirqd/2]
 21  root      0:00  [kworker/2:0]
 22  root      0:00  [kworker/2:0H]
 23  root      0:00  [cpuhp/3]
 24  root      0:00  [kworker/0:1-rcu]
 25  root      0:00  [migration/3]

```

- Uname -a: the command display the information about the currently running operating system and kernel. It was used to verify that the system was running the expected Linux kernel and ARM architecture. The output

confirmed that Linux kernel version 5.15.0 was running on the ARM-based QEMU system

- b. Mount: the command displays the filesystems currently mounted in the system. The output log was used to verify the mounting of the essential virtual filesystems required by the embedded Linux environment
- c. Ps: the ps command displays the processes currently running in the system. The command was used to verify that the userspace environment had been successfully initialized and the system processes were running. The init process is the first process that started by the Linux kernel and normally has PID 1. Thus, confirmed that the BusyBox userspace was operational after the kernel completed its boot process.

4. Theoretical Analysis & Technical Discussion

4.1 Comparison between arm-linux-gnueabi and arm-linux-gnueabihf

- **arm-linux-gnueabi (Soft-Float ABI):**
 - Floating-point function parameters are passed via general-purpose core integer registers (r0–r3).
 - Floating-point arithmetic is translated into software helper library routines (e.g., `__aeabi_fadd`).
 - Defaults to `-march=armv5te` with the dynamic linker path `/lib/ld-linux.so.3`.
- **arm-linux-gnueabihf (Hard-Float ABI):**
 - Floating-point parameters are passed directly via VFP hardware registers (d0–d7).
 - Arithmetic operations generate native assembly instructions (e.g., `vadd.f32`) executed directly on the physical FPU.
 - Defaults to `armv7-a` with VFPv3-D16, utilizing `/lib/ld-linux-armhf.so.3` as the dynamic linker.
- **Justification for Cortex-A9:**
 - The ARM Cortex-A9 core implements the ARMv7-A architecture equipped with an integrated VFPv3 floating-point unit.
 - Employing `gnueabihf` is mandatory to utilize hardware FPU execution, eliminate software emulation overhead, and avoid the performance penalty of marshaling data between core integer and VFP registers across function call boundaries. Furthermore, all standard modern

ARMv7-A embedded distributions and root filesystems adopt the `armhf` standard.

4.2 Memory Mapping: Versatile PB vs. Versatile Express

- **Memory Architecture Differences:**

- **Versatile PB:** A monolithic development board design where physical DRAM is mapped linearly starting at address `0x00000000`.
- **Versatile Express (VExpress):** Built upon a modular motherboard and CoreTile daughterboard architecture. The lower physical memory space is pre-allocated:
 - `0x00000000–0x0BFFFFFF`: Reserved for CS0–CS3 NOR Flash and SRAM.
 - `0x10000000`: Motherboard system registers and memory-mapped I/O peripherals (PL011 UART, MMCI, system timers).
 - `0x1E000000`: `PERIPHBASE` for the Cortex-A9 MPCore subsystem (Snoop Control Unit, Generic Interrupt Controller, private timers/watchdog).
 - Consequently, physical system DRAM is mapped to the window starting at `0x60000000`.

- **Load Address Constraints in `bootz 0x60100000 0x64000000 0x60000000`:**

- All binary images must reside within valid physical RAM ($\geq 0x60000000$).
- `0x60000000` (**DTB**): Placed at the base of DRAM due to its small footprint and early parsing requirements.
- `0x60100000` (**zImage**): Positioned 1 MB (`+0x00100000`) above the DRAM base. When executed, the self-extracting `zImage` inflates the uncompressed kernel image into `RAM_BASE + 0x8000` (`0x60008000`). Placing the compressed image at least 1 MB away prevents the decompression engine from overwriting its own compressed source stream.
- `0x64000000` (**initramfs**): Offset by 64 MB (`+0x04000000`) from the DRAM base to provide contiguous headroom for the uncompressed kernel image and its page tables, avoiding memory overlap during initialization.

- **Failure Modes and Debugging:**

- *Loading outside RAM*: U-Boot halts with `Bad Linux ARM zImage magic!`.
- *Kernel overwrites DTB*: Generates `Error: invalid dtb and unrecognized machine ID`.
- *Kernel overwrites initramfs*: Generates `Initramfs unpacking failed: invalid magic`, followed by `Kernel panic - not syncing: VFS: Unable to mount root fs / No working init found`.

- Memory collisions typically manifest as an unhandled CPU hang immediately following the message **Starting kernel** Debugging this requires compiling the kernel with **CONFIG_DEBUG_LL=y** and appending **earlyprintk** to the kernel **bootargs**.

4.3 Analysis of CPU Main ID Register (MIDR)

The 32-bit hardware register value **0x410FC090** reported during CPU initialization decodes into the following architectural fields:

- **Bits (Implementer):** **0x41** (ASCII 'A') → **ARM Limited**.
- **Bits (Variant):** **0x0** → Major silicon revision **r0**.
- **Bits (Architecture):** **0xF** → Escape code indicating that CPU architecture features are defined by the **CPUID feature registers** rather than a legacy numerical field. From ARMv7 onward, the kernel queries the **ID_PFR**, **ID_MMFR**, and **ID_ISAR** registers to identify processor capabilities, identifying the core as **ARMv7**.
- **Bits [15:4] (Primary Part Number):** **0xC09** → **Cortex-A9**.
- **Bits [3:0] (Revision):** **0x0** → Minor silicon revision patch **p0**.
- **Complete Identification:** **ARM Cortex-A9 r0p0, ARMv7-A**.
- **Architectural Reference (0xC0x Series):** **0xC08** = Cortex-A8, **0xC09** = Cortex-A9, **0xC0F** = Cortex-A15, **0xD03** = Cortex-A53. Validated at runtime via **/proc/cpuinfo** displaying **CPU part: 0xc09** and **CPU architecture: 7**.

4.4 Root Cause of Init Execution Failure

- **Mechanism of Failure in Minimal **initramfs**:**
 - At system bootstrap, **initramfs** is populated into a bare memory-backed root filesystem containing no dynamic runtime services.
 - A dynamically compiled ELF binary contains a **PT_INTERP** program header specifying the dynamic linker path (e.g., **/lib/ld-linux-armhf.so.3**). When this loader is missing from the filesystem, the kernel's **do_execve()** system call fails with:
 - Plaintext can't run '/sbin/init': No such file or directory
 - This message indicates that the executable interpreter specified inside the binary was not found, rather than **/sbin/init** itself being absent.
- **Benefits of Static Linking:**
 - Avoids toolchain dynamic library version mismatches.
 - Accelerates system startup by eliminating runtime symbol resolution for every invoked binary.
 - Decreases total image footprint by eliminating the overhead of bundling multi-megabyte shared libraries into a small ramdisk.
- **Dynamic Dependency Resolution Procedure:**
 - Identify binary requirements using toolchain utilities:

- Bash

`arm-linux-gnueabihf-readelf -l <binary> | grep interpreter`

`arm-linux-gnueabihf-readelf -d <binary> | grep NEEDED`

- *Mandatory Shared Libraries:*

1. `ld-linux-armhf.so.3` (Dynamic Linker/Loader)
2. `libc.so.6` (Core C standard runtime)
3. `libm.so.6` (Math runtime, required by math-oriented applets like `awk` and `expr`)
4. `libgcc_s.so.1` (GCC runtime exceptions)
5. `libcrypt.so.1` (Required if authentication applets like `login/passwd` are enabled)

- **Toolchain Library Locations:**

- Locate sysroot base path: `arm-linux-gnueabihf-gcc -print-sysroot`.
- Standard location: `/usr/arm-linux-gnueabihf/lib/`.
- `libgcc_s.so.1` location:
`/usr/lib/gcc-cross/arm-linux-gnueabihf/<version>/`.
- All libraries must be copied preserving symbolic links (`cp -a`) directly into `/lib/` within the rootfs (not `/lib/arm-linux-gnueabihf/`).

4.5 Kernel Architecture in SMP Mode

- **Architectural Modifications for SMP Operation:**

- **Synchronization:** Kernel spinlocks switch from single-core no-op branches to hardware atomic primitives (`ldrex/strex`) backed by explicit memory barriers (`dmb/dsb`), increasing kernel binary footprint by 3%–6%.
- **Snoop Control Unit (SCU):** Enabled at `PERIPHBASE (0x1E000000)` to maintain hardware cache coherency across per-core L1 data caches.
- **Interrupt Subsystem (GIC):** The GIC Distributor is configured globally, while each core independently initializes its private GIC CPU Interface. Software Generated Interrupts (SGIs) are registered for Inter-Processor Communication (IPI).
- **Per-Core Resources:** Initializes per-core private timers (`smp_twd`), instantiates distinct per-CPU memory structures, separate runqueues, and launches dedicated secondary idle tasks (`swapper/1`).

- **Secondary CPU Bring-Up Flow:**

- Boot CPU (Core 0) parses the Device Tree node `cpu@1` with property `enable-method = "arm,cortex-a9-smp"`.
- Core 0 writes the entry point address `secondary_startup` into the motherboard register `SYS_FLAGSSET (0x10000030)`.

- Core 0 issues a Software Generated Interrupt (SGI) to wake Core 1 from its low-power **wfi** (Wait For Interrupt) state in the holding pen.
- Core 1 wakes up, initializes its local MMU and Page Tables (**TTBR**), configures its private GIC interface and timer, and drops into its idle scheduling loop.
- **Primary SMP Boot Log Signatures:**
 - **CPU1: thread -1, cpu 1, socket 0, mpidr 80000001**
Significance: Absent in single-core mode. Proves Core 1 successfully exited the holding pen and transitioned to kernel execution. In **mpidr = 0x80000001**, Bit 31 confirms the Multiprocessing Extensions are active, and **Aff0 = 1** identifies CPU core 1 within cluster 0.
 - **SMP: Total of 2 processors activated (2048.00 BogoMIPS).**
Significance: Contrasts with the uniprocessor output (**1 processors activated (1024.00 BogoMIPS)**). Total BogoMIPS doubles because the kernel aggregates calibration delay loops from both active hardware threads.
 - **rcu: RCU restricting CPUs from NR_CPUS=4 to nr_cpu_ids=2**
Significance: Default configuration supports up to four Cortex-A9 cores (**NR_CPUS=4**), but the Read-Copy Update (RCU) subsystem scales down its internal data structures to match the QEMU-provisioned count (**nr_cpu_ids=2**).
- **Runtime Verification:**
 - Executing **cat /sys/devices/system/cpu/online** returns **0-1**.
 - Executing **ps** shows active per-core kernel worker threads: **[migration/1]** and **[ksoftirqd/1]**.

5. Troubleshooting & Problem Resolution

- **1. ncurses Library Detection Failure during BusyBox **make menuconfig**:**
 - *Root Cause:* The host Ubuntu environment uses an updated GCC toolchain enforcing stricter C23 standards (**-Werror=implicit-int**). Consequently, **check-lxdialog.sh**'s legacy **main()** syntax failed compilation, causing the script to mistakenly conclude that the **ncurses** development headers were missing from the system.
 - *Resolution:* Standardize function definitions in the configuration test script:
 - Bash sed -i 's/main()/int main(void)/g' scripts/kconfig/lxdialog/check-lxdialog.sh
- **2. Compiler Error: **linker input file not found: march=armv7-a**:**
 - *Root Cause:* The leading hyphen (-) was omitted in the BusyBox **Additional CFLAGS** configuration field. GCC interpreted **march=armv7-a** as an input object file or linker script rather than a compiler target option.

- *Resolution:* Correct the compiler flags directly in the BusyBox configuration file:
- Bash

```
CONFIG_EXTRA_CFLAGS="-march=armv7-a -mfpu=neon-vfpv3
-mfloat-abi=hard"
```

- **3. Flash Memory Boundary Warning during U-Boot `saveenv`:**

- *Root Cause:* When executing direct QEMU simulation via `-kernel u-boot`, U-Boot is loaded directly into RAM without an attached, sector-aligned writable physical NOR flash block device.
- *Resolution:* This warning is non-fatal emulator behavior. Runtime environment parameters (`bootargs`, `bootcmd`) reside intact in memory for the active session, as validated using `printenv`.